Organizing Your Repository and Building Generic Collections

Details

Weighting:	Part of Labs grade
Submission Instructions:	Push to GitHub
Due Date:	Tuesday, Oct. 13th

Overview

In this lab, you will complete two major tasks:

1. Reorganize your existing COMP 258 repository into a more maintainable package structure and create a launcher program capable of running any completed lab
2. Convert your linked list implementation into a generic collection that can store and compare different data types safely using Java generics

Throughout this process, you will reinforce concepts including inheritance, polymorphism, package organization, abstraction, and generics. **Important**: Do not create a new repository. Refactor your existing repository so Git can track all file additions, removals, and moves.

Learning Outcomes

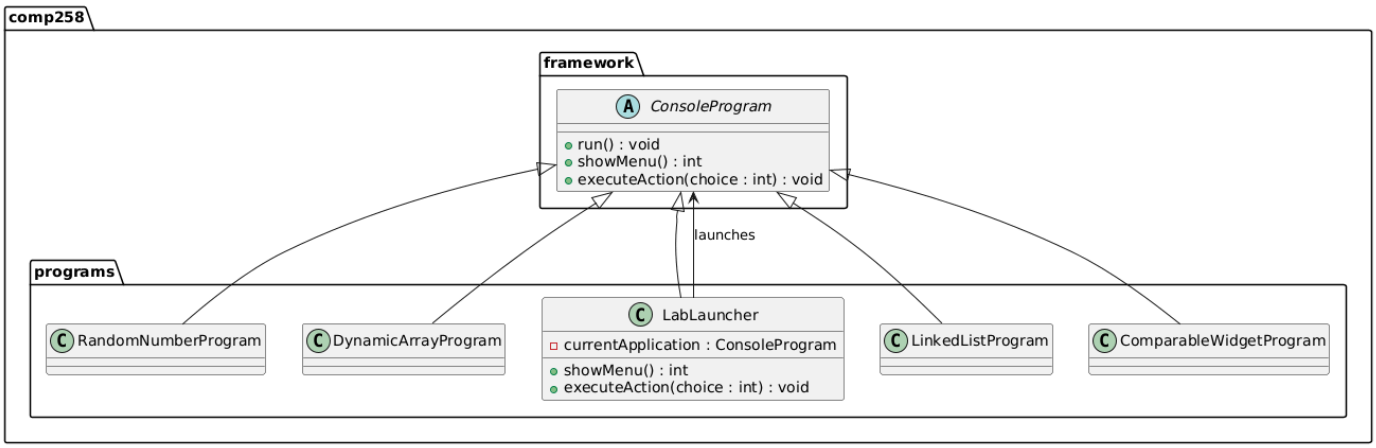
- Organize Java projects using packages and imports
- Refactor existing code without changing functionality
- Reuse functionality through inheritance
- Build applications from a shared framework
- Use polymorphism to execute different program types
- Create generic data structures using type parameters
- Apply a Comparable<T> constraint to support ordering and comparison
- Explain why generic type constraints can provide behavior that Object alone cannot

Part 1: Repository Organization

- As your projects grow, related classes should be grouped according to their responsibilities. Create the following package structure:

Package	Purpose	Contents
comp258	Main entry point (Driver)	Driver
comp258.framework	Shared framework classes	ConsoleProgram
comp258.programs	User-facing applications	LabLauncher, RandomNumberProgram, ArrayManagerProgram, LinkedListProgram, etc.
comp258.collections	Collection implementations	LinkedList, ListNode, ArrayManager, etc.
comp258.exceptions	Custom exceptions	OutOfBoundsException, etc.
comp258.labs	Experimental code & practice exercises	Standalone exercises and experiments

1. Create a package named: comp258.framework
    a. Move your existing ConsoleProgram class into this package
2. Create a new class named: LabLauncher
    a. Place it inside comp258.programs
    b. The launcher should extend ConsoleProgram
    c. It allows the user to choose which completed lab to run



d. This class has one private variable called currentProgram of type ConsoleProgram that holds an instance of a ConsoleProgram child class

3. Implement the two abstract methods of the framework class:

```
@Override
public int showMenu() {
    System.out.printf("""

            COMP 258 Labs

                1  Random Number App
                2  Array Manager App
                3  Linked List App
                4  Generic List App

                %d  exit

            """, QUIT_OPTION);
    return readInt("Choose lab application: ");
}


@Override
public void executeAction(int choice) {
    currentApplication = switch (choice) {
        case 1 -> new RandomNumberProgram();
        case 2 -> new DynamicArrayProgram();
        case 3 -> new LinkedListProgram();
        case 4 -> new ComparableWidgetProgram();
        default -> new HelloWorldProgram();
    };
    currentApplication.run();
}
```

Use @Override to implement the hook methods as well.

4. Create (or update) your Driver class so that it launches the application launcher.

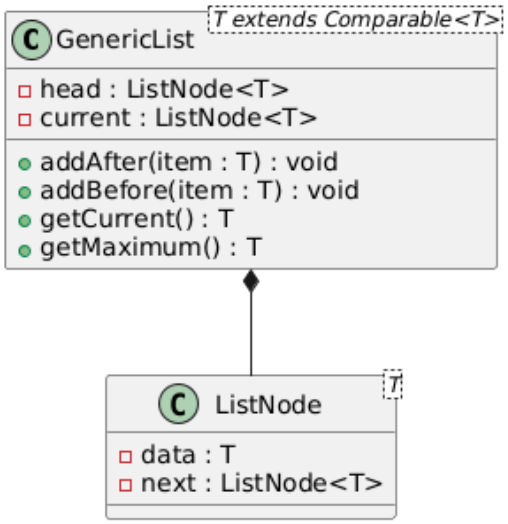
```
Public class Driver {
	public static void main(String[] args) {
		ConsoleProgram app = new LabLauncher();
		app.run();
	}
}
```

Part 2: Generic Lists

5. Create a new generic version of your linked list. Rather than storing Object values, your collection should support any comparable data type.
   a. Your list must use a bounded type parameter:

```
public class GenericList<T extends Comparable<T>>
```

   b. This constraint guarantees that every stored item can be compared to another item of the same type.



Why Use Comparable?

The original list stored values as `Object`. Imagine if you had:

```
Object a = "Apple";
Object b = "Banana";
```

Although both objects are actually strings, the compiler only knows they are of type `Object`, which means you aren't allowed to call what seems like a sensible:

```
a.compareTo(b);
```

The `Object` class does not define a `compareTo()` method, but with:

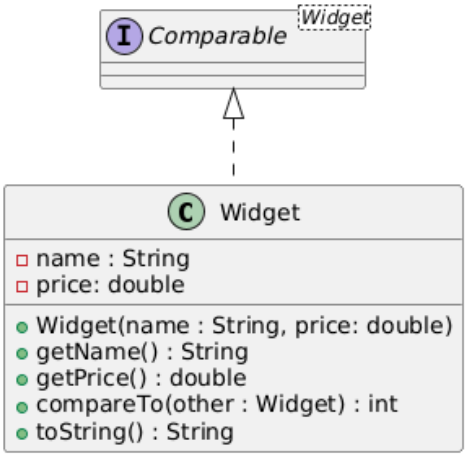
```
T extends Comparable<T>
```

the compiler guarantees that `compareTo()` exists, making ordering operations possible.

6. Add a method that finds the largest item in the list. This method must:
    a. Return the highest value as decided by `compareTo()`
        i. Return null if the list is empty
    b. Move the current position to the maximum item
    c. Return null if the list is empty

Part 3: Testing

7. Demonstrate your generic list with multiple types, including Integer, String, and Widget. Recall Widget must extend Comparable<Widget> & implement compareTo
    a. Widgets have the following fields
        i. String name
        ii. double price



Note: `compareTo()` should return a negative value, zero, or a positive value based on Widget price.

8. Program should start with an empty list and handle the following:
    a. print list
    b. get max
    c. get current
    d. advance
    e. remove

9. Create several Widget objects and verify that
    a. items are inserted correctly
    b. comparisons work correctly
    c. max values are found correctly
    d. current position moves to the maximum item

GitHub Repo Requirements

- All classes have been moved into appropriate packages
- ConsoleProgram is located in comp258.framework
- LabLauncher can launch completed labs from the Driver
- Your generic list compiles without warnings
- getMaximum() works for all tested data types
- Tests have been completed for Integer, String, and Widget
- Changes have been pushed to GitHub